

Logical Reasoning for Planning

Warehouse Robot Planning Agent

Name: Sidh Virmani **Course:** Artificial Intelligence

GitHub: https://github.com/Sidh-Virmani/2024A7PS0520G_AI_HandsOn

1. Planning Problem

The planning problem is represented as **(I, A, G)**, where **I** is the initial state, **A** is the set of available actions, and **G** is the goal.

Initial state: $I = \{At(Robot,A), At(Package,A)\}$

Goal: $G = \{At(Package,C)\}$

Warehouse connections: $A \leftrightarrow B \leftrightarrow C$

Action	Preconditions	Effects
Move(X,Y)	At(Robot,X), with X and Y connected	remove At(Robot,X); add At(Robot,Y)
PickUp(Package,L)	At(Robot,L), At(Package,L), and not Holding(Package)	remove At(Package,L); add Holding(Package)
Drop(Package,L)	At(Robot,L), Holding(Package)	remove Holding(Package); add At(Package,L)

Initially applicable actions: Move(A,B) and PickUp(Package,A). PickUp(Package,A) is applicable because the robot and package are both at A and the robot is not already holding the package. Drop(Package,C) is not applicable because the robot is not at C and is not holding the package.

2. Manually Constructed Plan

Plan: PickUp(Package,A) -> Move(A,B) -> Move(B,C) -> Drop(Package,C)

State	Facts
S0	At(Robot,A), At(Package,A)
S1	At(Robot,A), Holding(Package)
S2	At(Robot,B), Holding(Package)
S3	At(Robot,C), Holding(Package)
S4	At(Robot,C), At(Package,C)

The final state contains At(Package,C), so the goal is satisfied.

3. Prompt Used with the LLM

Implement the warehouse logical planning problem from the attached lab sheet in Python. Represent states as sets of propositions and actions using positive/negative preconditions and positive/negative effects. Use Breadth-First Search (BFS) to find a valid sequence of actions that reaches the specified goal and report when no plan exists. Print the action sequence and the state reached after every action, and include Tests A, B, and C described in the lab. Add detailed comments explaining what every important function and section of the code is doing.

The same prompt is stored in **prompt.txt**. The generated and tested implementation is stored in **Code.py**.

4. Implementation Summary

Each action stores a name, positive preconditions, negative preconditions, positive effects, and negative effects. An action is applicable only when every positive precondition is present and every negative precondition is absent.

Applying an action first removes its negative effects and then adds its positive effects. Breadth-First Search uses a FIFO queue to explore reachable states. A visited set prevents repeated states and cycles. Planning terminates successfully when every goal proposition is contained in the current state. If the queue becomes empty first, the program reports **No plan found**.

Planning idea	Where it appears in Code.py
Preconditions	Action.is_applicable()
Effects	Action.apply()
Goal test	goal.issubset(current_state)
BFS	deque queue, popleft(), and successor expansion

5. Test Results

Test A - Solvable Problem

Initial state: {At(Robot,A), At(Package,A)}

Goal: {At(Package,C)}

Plan found: Yes

Resulting plan: Pickup(Package,A) -> Move(A,B) -> Move(B,C) -> Drop(Package,C)

Validity: Valid. Every action satisfies its preconditions when executed, and the final state contains At(Package,C).

Test B - Impossible Problem

Initial state: {At(Robot,A), At(Package,A)}

Goal: {At(Package,C)}

Modification: All Pickup actions are removed.

Plan found: No

Resulting plan: No plan found

Validity: Correct. The robot may move, but the package can never be carried to C.

Test C - Irrelevant Actions

Initial state: {At(Robot,A), At(Package,A)}

Goal: {At(Package,C)}

Modification: An additional direct robot movement from A to C is added.

Plan found: Yes

Resulting plan: The planner still returns a plan that actually transports and drops the package at C.

Validity: Valid. Reaching C with only the robot does not satisfy the goal, because the goal is At(Package,C), not At(Robot,C).

6. Logic and Search

Completed flow: Current state -> Check action preconditions -> If satisfied, apply the action -> Generate successor state -> Search over alternatives -> Goal test.

Logical reasoning determines whether an action is applicable and how the state changes after that action. Search determines which applicable action sequences are explored. In short: **logic determines what is possible; search determines what to try.**

7. Think About It

Why is an action not applicable merely because it is listed? Because its preconditions must also be satisfied in the current state.

Where do the main planning ideas appear in the program? Preconditions are checked in `is_applicable()`, effects are applied in `apply()`, the goal is tested with `goal.issubset(current_state)`, and BFS explores alternatives using a FIFO queue.

How do logic and search work together? Logic filters legal transitions and computes successor states; BFS explores those valid transitions until a goal state is reached.

Which should be trusted more: an LLM explanation or independently executed state transitions? The independently executed transitions, because they directly evaluate the implemented preconditions, effects, and goal test rather than only describing them in prose.

8. Reflection Questions

1. Why is it useful to specify action preconditions and effects before asking an LLM to write the planner?

They define precisely when actions are legal and how they change the state, reducing ambiguity in the generated implementation.

2. Give an example of an error that could occur if the planner failed to check an action's preconditions.

It could drop the package at C when the robot is not at C or is not holding the package.

3. Why is a plan that looks reasonable not necessarily a valid plan?

Every action must have its preconditions satisfied in the exact state in which that action is executed.

4. What did the LLM contribute to the implementation?

It generated the Python structure, action representation, BFS planner, test setup, and explanatory comments from the supplied specification.

5. What did you have to verify independently?

The program was executed and the returned plans, state transitions, goal condition, and impossible case were checked against the lab requirements.

6. In this laboratory, where is logical reasoning being used?

It is used to check action preconditions and to apply positive and negative effects to obtain successor states.

7. How is planning related to the search algorithms studied in the previous module?

States act as search nodes, actions act as transitions, the initial state is the start node, and states satisfying the goal are goal nodes.

9. LLM Assistance and Verification

ChatGPT was used to generate and explain the Python implementation from the planning specification. The program was then executed and checked independently using Tests A, B, and C. The report and code therefore follow the workflow: **Understand -> Specify -> Generate -> Execute -> Verify.**